1. Problem Title & Link

Title: **LeetCode 303: Range Sum Query – Immutable**

Link: https://leetcode.com/problems/range-sum-query-immutable/

2. Problem Statement (Short Summary)

You are given an integer array nums.

You need to answer multiple queries where each query asks:

What is the sum of elements from index left to index right (inclusive)?

Instead of calculating the sum every time, we should **optimize the process so queries can be answered very quickly**.

This problem introduces the **Prefix Sum technique**.

3. Examples (Input → Output)

Example 1

Input

nums = [-2,0,3,-5,2,-1]

Query

sumRange(0,2)

Output

1

Explanation

-2 + 0 + 3 = 1

Example 2

Input

sumRange(2,5)

Output

-1

Explanation

3 + (-5) + 2 + (-1) = -1

Example 3

Input

sumRange(0,5)

Output

-3

Explanation

-2 + 0 + 3 - 5 + 2 - 1 = -3

4. Constraints

$1 \le nums.length \le 10^4$

$-10^5 \le nums[i] \le 10^5$

$0 \le left \le right < nums.length$

At most 10^4 queries

Observations:

- Multiple queries
- Need **fast query response**
- Recomputing sums repeatedly is inefficient

5. Core Concept (Pattern / Topic)

Prefix Sum

Why?

Prefix sums allow us to compute range sums in **O(1)** time after preprocessing.

6. Thought Process (Step-by-Step Explanation)

Step 1 — Brute Force

For each query:

```
sum = 0
for i in range(left, right):
    sum += nums[i]
```

Time Complexity per query:

O(n)

If we have many queries:

O(n * q)

This becomes inefficient.

Step 2 — Optimization Idea

We compute **prefix sums**.

Prefix sum definition:

prefix[i] = sum of elements from index 0 to i

Example:

nums = [-2,0,3,-5,2,-1]

Prefix array:

index:	0	1	2	3	4	5
nums:	-2	0	3	-5	2	-1

prefix: -2 -2 1 -4 -2 -3

Step 3 — Range Sum Formula

To compute:

sum(left → right)

We use:

prefix[right] - prefix[left-1]

Special case:

left == 0

Then:

prefix[right]

7. Visual / Intuition Diagram (ASCII)

Array

index: 0 1 2 3 4 5

nums: -2 0 3 -5 2 -1

Prefix

prefix[i] = sum(0 → i)

prefix:

[-2, -2, 1, -4, -2, -3]

Query example

sumRange(2,5)

prefix[5] - prefix[1]

-3 - (-2) = -1

8. Pseudocode (Language Independent)

```
create prefix array

prefix[0] = nums[0]

for i from 1 to n-1:
    prefix[i] = prefix[i-1] + nums[i]
```

```
function sumRange(left, right):

    if left == 0:
        return prefix[right]

    return prefix[right] - prefix[left-1]
```

9. Code Implementation

Python

```
class NumArray:

    def __init__(self, nums):
        self.prefix = [0]*len(nums)

        self.prefix[0] = nums[0]

        for i in range(1, len(nums)):
            self.prefix[i] = self.prefix[i-1] + nums[i]

    def sumRange(self, left, right):

        if left == 0:
            return self.prefix[right]

        return self.prefix[right] - self.prefix[left-1]
```

Java

```
class NumArray {

    int[] prefix;

    public NumArray(int[] nums) {

        prefix = new int[nums.length];

        prefix[0] = nums[0];
```

```
        for(int i = 1; i < nums.length; i++) {
            prefix[i] = prefix[i-1] + nums[i];
        }
    }

    public int sumRange(int left, int right) {

        if(left == 0)
            return prefix[right];

        return prefix[right] - prefix[left-1];
    }
}
```

10. Time & Space Complexity

Preprocessing

O(n)

Query Time

O(1)

Space Complexity

O(n)

For storing prefix array.

11. Common Mistakes / Edge Cases

1. Forgetting the **left = 0 case**
2. Wrong prefix index usage
3. Using prefix[right] - prefix[left] instead of prefix[left-1]
4. Not initializing prefix correctly
5. Empty array handling

12. Detailed Dry Run

Example

nums = [-2,0,3,-5,2,-1]

Prefix array

i	nums[i]	prefix[i]
0	-2	-2

1	0	-2
2	3	1
3	-5	-4
4	2	-2
5	-1	-3

Query

sumRange(2,5)

Calculation

prefix[5] - prefix[1]

-3 - (-2) = -1

Answer

-1

13. Common Use Cases (Real-Life / Interview)

Prefix sums are widely used in:

- Financial analytics (cumulative profit)
- Stock market moving sums
- Subarray sum problems
- Image processing
- Range queries in databases

14. Common Traps (Important!)

1. Forgetting to handle left = 0
2. Incorrect prefix array indexing
3. Misunderstanding inclusive range
4. Not preprocessing before queries

15. Builds To (Related LeetCode Problems)

Prefix Sum progression:

LC 303 Range Sum Query

LC 560 Subarray Sum Equals K

LC 523 Continuous Subarray Sum

LC 974 Subarray Sums Divisible by K

LC 1248 Count Nice Subarrays

16. Alternate Approaches + Comparison

Approach	Complexity	Notes
Brute Force	O(n) per query	Slow
Prefix Sum	O(1) query	Optimal

17. Why This Solution Works

Prefix sum stores the **cumulative sum of elements up to each index**.

By subtracting two prefix values, we remove the unwanted portion and directly obtain the required subarray sum.

18. Variations / Follow-Up Questions

Interview variations:

1. Array updates allowed → use **Segment Tree**
2. 2D matrix range sum → **2D Prefix Sum**
3. Count subarrays with sum = K
4. Count subarrays divisible by K